

Improve Python Code by Using a Profiler



The `line_profiler` gives a line-by-line analysis of the Python code and can thus identify bottlenecks that slow down the execution of a program. By making modifications to the code based on the results of this profiler, developers can improve the code and refine the program.

Have you ever wondered which module is slowing down your Python program and how to optimise it? Well, there are 'profilers' that can come to your rescue.

Profiling, in simple terms, is the analysis of a program to measure the memory used by a certain module, frequency and duration of function calls, and the time complexity of the same. Such profiling tools are termed profilers. This article will discuss the `line_profiler` for Python.

Installation

Installing pre-requisites: Before installing `line_profiler` make sure you install these pre-requisites:

a) For Ubuntu/Debian-based systems (recent versions):

```
sudo apt-get install mercurial python python3 python-pip
python3-pip Cython Cython3
```

b) For Fedora systems:

```
sudo yum install -y mercurial python python3 python-pip
```



Note: 1. I have used the '-y' argument to automatically install the packages after being tracked by the yum installer.

2. Mac users can use *Homebrew* to install these packages.

Cython is a pre-requisite because the source releases require a C compiler. If the Cython package is not found or is too old in your current Linux distribution version, install it by running the following command in a terminal:

```
sudo pip install Cython
```



Note: Mac OS X users can install Cython using *pip*.